

LAPORAN PRAKTIKUM W2

Object, Class & Encapsulation in Java Part 2

Diajukan untuk memenuhi salah satu tugas praktikum mata kuliah Pemrograman Berorientasi Objek.



Disusun Oleh:

Nama : R. Neysa Rahma Velda
NIM : 251511057
Kelas : D3 - 2B

Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026

1. Deskripsi Aplikasi

Aplikasi yang dibangun merupakan sistem manajemen cuti karyawan berbasis teks (Command Line Interface) untuk perusahaan PT. Stanford English School. Program ini dirancang menggunakan konsep Pemrograman Berorientasi Objek (PBO) tingkat lanjut, yang meliputi penerapan package, static field & method, relasi antar-kelas (Aggregation dan Dependency), serta pemaketan aplikasi menjadi file executable (.jar).

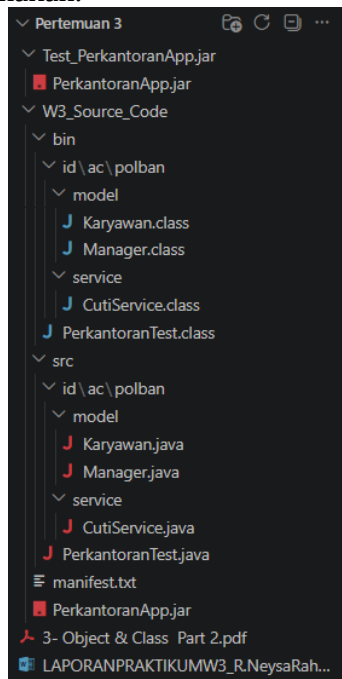
Program ini terbagi menjadi beberapa komponen utama dengan pembagian tugas (Separation of Concerns) yang jelas. Kelas Karyawan berperan sebagai model untuk menyimpan atribut data individu karyawan seperti nama, NIP, divisi, dan sisa cuti, serta mengelola data global melalui static constant dan static counter. Kelas Manager berperan sebagai pengelola yang memiliki relasi agregasi terhadap objek Karyawan melalui struktur koleksi berupa List. Kelas CutiService bertindak sebagai layanan bisnis yang memproses logika pengajuan cuti dengan menerapkan relasi dependency. Terakhir, kelas PerkantoranTest berfungsi sebagai driver class utama untuk menginisialisasi objek, menghubungkan relasi, dan menjalankan skenario pengujian.

2. Struktur Package dan Class

a. Pada program digunakan dua package utama sesuai instruksi praktikum, yaitu:

- id.ac.polban.model
- id.ac.polban.service

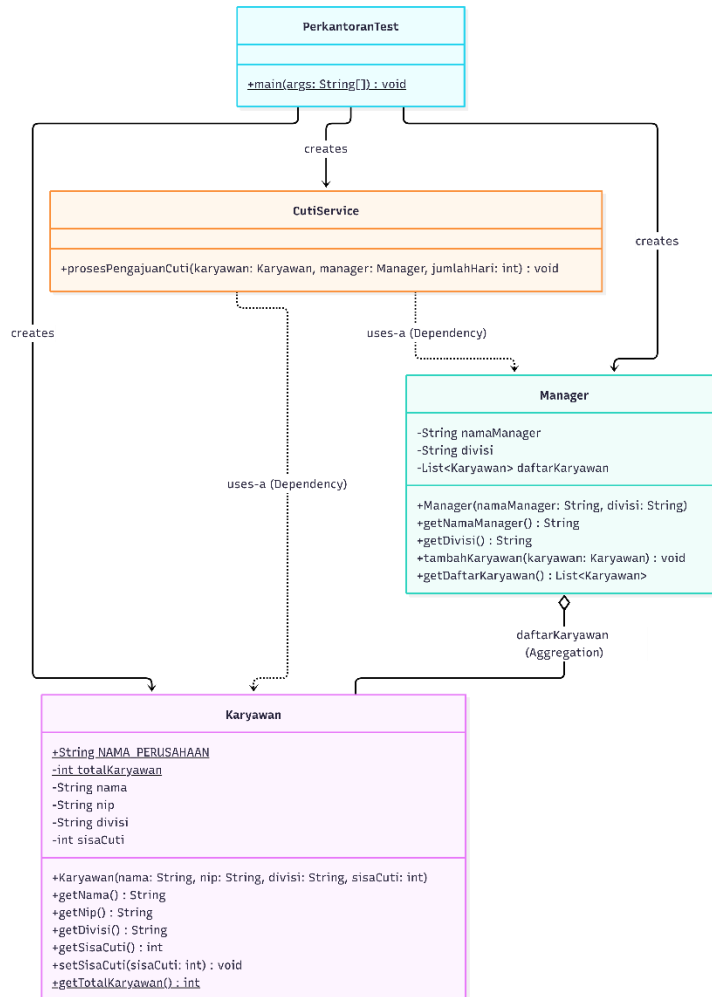
b. Struktur roject yang digunakan:



Package digunakan untuk mengelompokkan class berdasarkan tanggung jawabnya. Pada program ini, Karyawan dan Manager ditempatkan pada package id.ac.polban.model, sedangkan CutiService ditempatkan pada package id.ac.polban.service. Penggunaan package membantu pengelolaan kode menjadi lebih terstruktur dan modular.

3. Class Diagram

Class diagram digunakan untuk menggambarkan struktur class dan hubungan antar-class pada aplikasi.



Relasi pertama yang diterapkan adalah Aggregation antara kelas Manager dan kelas Karyawan. Hubungan ini terbentuk karena kelas Manager memiliki atribut berupa daftar koleksi (`List<Karyawan> daftarKaryawan`). Objek Karyawan diciptakan secara independen di dalam kelas penguji **PerkantoranTest**, kemudian diregistrasikan ke dalam manajer melalui method `tambahKaryawan()`. Pola ini menunjukkan hubungan has-a, di mana objek Karyawan tetap dapat berdiri sendiri meskipun objek Manager-nya tidak ada.

Relasi kedua adalah Dependency antara kelas **CutiService** terhadap **Karyawan** dan **Manager**. Kelas **CutiService** bergantung pada kedua kelas tersebut karena objek **Karyawan** dan **Manager** diterima sebagai parameter eksplisit di dalam method `prosesPengajuanCuti(Karyawan karyawan, Manager manager, int jumlahHari)`. Hubungan ini bersifat uses-a, di mana layanan penunjang memerlukan objek-objek tersebut hanya saat proses pengajuan cuti berlangsung.

4. Penerapan Static Field dan Static Method

Pada class Karyawan, terdapat dua static field dan satu static method.

a. Static Field dan Static Method

```
public static final String NAMA_PERUSAHAAN = "PT. Stanford  
English School";
```

Pada kelas Karyawan, terdapat beberapa implementasi elemen static untuk kebutuhan data global dan pencatatan terpusat. Atribut NAMA_PERUSAHAAN dideklarasikan menggunakan public static final String agar bernilai konstanta global yang melekat pada level kelas, sehingga dapat diakses langsung melalui Karyawan.NAMA_PERUSAHAAN tanpa harus membuat instansiasi objek terlebih dahulu.

Selain itu, terdapat atribut totalKaryawan yang bertindak sebagai static counter untuk menghitung jumlah total objek Karyawan yang berhasil diinisialisasi melalui constructor (totalKaryawan++). Untuk mengakses nilai penghitung tersebut secara aman, disediakan sebuah static method bernama getTotalKaryawan(), yang memungkinkan pemanggilan data secara langsung via nama kelas (Karyawan.getTotalKaryawan()).

b. Static Counter

Objek Karyawan dan Manager dibuat menggunakan constructor.

```
Manager manager1 =  
    new Manager("Pak Raden", "Divisi IT");  
  
Karyawan karyawan1 =  
    new Karyawan(  
        "Neysa Rahma",  
        "NIP-101",  
        "Divisi IT",  
        12  
    );
```

Constructor digunakan untuk memberikan nilai awal kepada attribute ketika objek dibuat. Constructor merupakan method yang otomatis dijalankan ketika sebuah class diinstansiasi dan memiliki nama yang sama dengan nama class.

c. Penerapan Object Construction dan Constructor

Inisialisasi objek di dalam aplikasi ini memanfaatkan constructor guna memastikan setiap objek langsung membawa nilai atribut yang valid sejak pertama kali diciptakan. Pada kelas Karyawan, constructor menerima empat parameter utama berupa nama, NIP, divisi, dan sisa cuti, yang kemudian langsung dimasukkan ke dalam atribut

internal menggunakan keyword this sekaligus menaikkan nilai penghitung totalKaryawan. Sementara itu, constructor pada kelas Manager menerima parameter nama manager dan divisi, sekaligus menginisialisasi wadah penampung daftar karyawan menggunakan new ArrayList<>().

6. Source Code

a. Karyawan.java

```
b. import id.ac.polban.model.Karyawan;
c. import id.ac.polban.model.Manager;
d. import id.ac.polban.service.CutiService;
e.
f. public class PerkantoranTest {
g.     public static void main(String[] args) {
h.
i.         System.out.println(
j.             "=== SISTEM MANAJEMEN CUTI "
k.             + Karyawan.NAMA_PERUSAHAAN
l.             + " ==="
m.         );
n.
o.         System.out.println("-----");
p.
q.         Manager manager1 =
r.             new Manager("Pak Raden", "Divisi IT");
s.
t.         Karyawan karyawan1 =
u.             new Karyawan(
v.                 "Neysa Rahma",
w.                 "NIP-101",
x.                 "Divisi IT",
y.                 12
z.             );
aa.
bb.         Karyawan karyawan2 =
cc.             new Karyawan(
dd.                 "Maria Magdalena",
ee.                 "NIP-102",
ff.                 "Divisi IT",
gg.                 3
hh.             );
ii.
jj.         // Implementasi Aggregation
kk.         manager1.tambahKaryawan(karyawan1);
ll.         manager1.tambahKaryawan(karyawan2);
mm.
```

```

nn.         System.out.println(
oo.             "Total Karyawan Terdaftar: "
pp.             + Karyawan.getTotalKaryawan()
qq.         );
rr.
ss.         System.out.println("-----");
tt.
uu.         // Membuat objek service untuk memproses cuti
vv.         CutiService layananCuti = new CutiService();
ww.
xx.         // Implementasi Dependency
yy.         layananCuti.prosesPengajuanCuti(
zz.             karyawan1,
aaa.             manager1,
bbb.             5
ccc.         );
ddd.
eee.         System.out.println("-----");
fff.
ggg.             layananCuti.prosesPengajuanCuti(
hhh.                 karyawan2,
iii.                 manager1,
jjj.                 5
kkk.             );
lll.     }
mmm. }

```

Kelas CutiService berfungsi sebagai pengendali logika layanan bisnis untuk proses pengajuan cuti. Kelas ini tidak menyimpan data objek karyawan maupun manager sebagai atribut global, melainkan menerimanya langsung sebagai parameter method prosesPengajuanCuti(). Di dalam method tersebut, dilakukan evaluasi perbandingan antara jumlah hari cuti yang diajukan dengan sisa kuota cuti karyawan; jika mencukupi maka sistem akan menyetujui dan mengurangi sisa cuti, sebaliknya sistem akan memberikan status penolakan.

d. Manager.java

```

e. package id.ac.polban.model;
f.
g. import java.util.ArrayList;
h. import java.util.List;
i.
j. public class Manager {
k.     private final String namaManager;
l.     private final String divisi;
m.     private final List<Karyawan> daftarKaryawan;
n.

```

```

o.     public Manager(String namaManager, String divisi) {
p.         this.namaManager = namaManager;
q.         this.divisi = divisi;
r.         this.daftarKaryawan = new ArrayList<>();
s.     }
t.
u.     public String getNamaManager() {
v.         return namaManager;
w.     }
x.
y.     public String getDivisi() {
z.         return divisi;
aa.    }
bb.
cc.     public void tambahKaryawan(Karyawan karyawan) {
dd.         daftarKaryawan.add(karyawan);
ee.     }
ff.
gg.     public List<Karyawan> getDaftarKaryawan() {
hh.         return daftarKaryawan;
ii.    }
jj.}

```

c. CutiService.kava

```

d. package id.ac.polban.service;
e.
f. import id.ac.polban.model.Karyawan;
g. import id.ac.polban.model.Manager;
h.
i. public class CutiService {
j.
k.     // Method untuk memproses pengajuan cuti (Menerapkan Dependency ke
    Karyawan dan Manager)
l.     public void prosesPengajuanCuti(Karyawan karyawan, Manager manager,
    int jumlahHari) {
m.         System.out.println(karyawan.getNama() + " mengajukan cuti " +
    jumlahHari + " hari kepada Manager " + manager.getNamaManager());
n.
o.         if (jumlahHari <= karyawan.getSisaCuti()) {
p.             karyawan.setSisaCuti(karyawan.getSisaCuti() - jumlahHari);
q.             System.out.println("Hasil: DISETUJUI. Sisa cuti " +
    karyawan.getNama() + " sekarang: " + karyawan.getSisaCuti() + "
    hari.");
r.         } else {

```

```

s.         System.out.println("Hasil: DITOLAK. Sisa cuti " +
    karyawan.getNama() + " tidak mencukupi (sisa: " +
    karyawan.getSisaCuti() + " hari).");
t.     }
u.     }
v. }

```

d. PerkatantoranTest.java

```
import id.ac.polban.model.Karyawan;
```

```
import id.ac.polban.model.Manager;
```

```
import id.ac.polban.service.CutiService;
```

```

public class PerkantoranTest {
    public static void main(String[] args) {

        System.out.println(
            "=== SISTEM MANAJEMEN CUTI "
            + Karyawan.NAMA_PERUSAHAAN
            + " ==="
        );

        System.out.println("-----");

        Manager manager1 =
            new Manager("Pak Raden", "Divisi IT");

        Karyawan karyawan1 =
            new Karyawan(
                "Neysa Rahma",
                "NIP-101",
                "Divisi IT",
                12
            );

        Karyawan karyawan2 =
            new Karyawan(
                "Maria Magdalena",
                "NIP-102",
                "Divisi IT",
                3
            );

        // Implementasi Aggregation
        manager1.tambahKaryawan(karyawan1);
    }
}

```

```

manager1.tambahKaryawan(karyawan2);

System.out.println(
    "Total Karyawan Terdaftar: "
    + Karyawan.getTotalKaryawan()
);

System.out.println("-----");

// Membuat objek service untuk memproses cuti
CutiService layananCuti = new CutiService();

System.out.println("\n--- KASUS 1: Pengajuan Cuti (Disetujui) ---");
// Implementasi Dependency (Kasus 1)
layananCuti.prosesPengajuanCuti(
    karyawan1,
    manager1,
    5
);

System.out.println("\n--- KASUS 2: Pengajuan Cuti (Ditolak) ---");
// Implementasi Dependency (Kasus 2)
layananCuti.prosesPengajuanCuti(
    karyawan2,
    manager1,
    5
);
}
}

```

Kelas PerkantoranTest bertindak sebagai titik masuk utama (driver class) untuk menjalankan aplikasi. Di dalam method main, program menginisialisasi objek Manager serta dua objek Karyawan dengan kuota cuti awal yang berbeda. Selanjutnya, objek-objek karyawan tersebut diregistrasikan ke dalam manajer untuk menerapkan relasi agregasi. Objek CutiService kemudian diinstansiasi guna mengeksekusi dua skenario pengujian kasus cuti secara berurutan, yang masing-masing merepresentasikan kondisi disetujui dan ditolak.

7. Hasil Menjalankan Program

```

D:\KULIAH\Semester 3\PBO\PBO Praktek\Pertemuan 3> cmd /C ""C:\Pr
ppData\Roaming\Code\User\workspaceStorage\0116ef832b76704552a721
=== SISTEM MANAJEMEN CUTI PT. Stanford English School ===
-----
Total Karyawan Terdaftar: 2
-----
Neysa Rahma mengajukan cuti 5 hari kepada Manager Pak Raden
Hasil: DISETUJUI. Sisa cuti Neysa Rahma sekarang: 7 hari.
-----
Maria Magdalena mengajukan cuti 5 hari kepada Manager Pak Raden

```

8. Proses Build dan Packaging JAR

Tahapan pembuatan file aplikasi terkompilasi dalam bentuk arsip jar dilakukan melalui perintah terminal sebagai berikut:

1. Membuat direktori penampung kelas hasil kompilasi dengan mengetikkan perintah `mkdir bin`.
2. Melakukan kompilasi seluruh file sumber kode Java ke dalam folder `bin` menggunakan perintah `javac -d bin src\id\ac\polban\model*.java src\id\ac\polban\service*.java src\PerkantoranTest.java`.
3. Melakukan pemaketan menjadi file arsip jar berdasarkan konfigurasi pada berkas `manifest.txt` dengan perintah `jar cfm PerkantoranApp.jar manifest.txt -C bin ..`.
4. Menjalankan aplikasi mandiri berbasis jar menggunakan perintah `java -jar PerkantoranApp.jar`.

9. Project untuk Menjalankan JAR

Untuk memastikan bahwa file jar bersifat mandiri dan dapat berjalan tanpa kode sumber utama, file `PerkantoranApp.jar` diuji pada direktori project terpisah bernama `Test_PerkantoranApp.jar`. Hasil eksekusi perintah `java -jar PerkantoranApp.jar` terbukti menghasilkan keluaran terminal yang sama persis seperti ketika dijalankan langsung dari direktori sumber kode utama.

10. Permasalahan yang Dihadapi dan Solusi

- Kendala 1 mengenai format path kompilasi dimana pada awal pengujian, perintah `javac` sempat mengalami kendala akibat struktur direktori path Windows yang belum sesuai. Solusinya adalah dengan menyesuaikan kembali argumen path relatif secara tepat mengarah ke masing-masing file kelas di dalam folder `src`.
- Kendala 2 mengenai direktori `bin` yang belum tersedia. Ketika proses kompilasi sempat tertahan karena folder tujuan `-d bin` belum terbentuk otomatis pada direktori lokal. Solusinya adalah menjalankan perintah pembuatan folder `mkdir bin` terlebih dahulu sebelum melakukan proses kompilasi.

11.Lesson Learnt

Praktikum ini memberikan pemahaman mendalam mengenai penerapan pilar-pilar Pemrograman Berorientasi Objek di Java, khususnya:

1. Hubungan antar kelas, di mana mahasiswa dapat memahami perbedaan implementasi relasi aggregation berupa hubungan kepemilikan koleksi daftar karyawan pada Manager, serta relasi dependency berupa ketergantungan objek parameter pada method CutiService.
2. Penggunaan keyword static untuk memanfaatkan static constant pada nilai global tetap dan static counter guna pelacakan pembuatan objek secara terpusat.
3. Modularitas package untuk mengorganisasikan kelas ke dalam package fungsional model dan service demi kerapian serta penerapan prinsip enkapsulasi yang baik.
4. Packaging aplikasi untuk menguasai proses kompilasi manual menggunakan perintah javac, pembuatan arsip jar, hingga pengujian aplikasi terkompilasi di lingkungan terpisah.

Lampiran :

https://github.com/Neysavelda/W3_R.NeysaRahmaVelda_251511057_PBO